

Что такое массивы

Массив (array) – это коллекция фиксированной длины, содержащая элементы одного типа. Длина массива определяется при его создании и не может быть изменена. Элементы массива могут быть любого типа, включая другие массивы и структуры.

```
var arr [5]int // массив из 5 целых чисел, по умолчанию заполнен нулями
```

Создание массива

Создание массива может включать или не включать инициализацию значений:

- Пустой массив:

```
var arr [5]int // массив из 5 целых чисел, по умолчанию заполнен нулями
```

- Инициализация массива с начальными значениями:

```
arr2 := [3]string{"a", "b", "c"}
```

- Автоматическое определение длины массива:

```
arr3 := [...]int{1, 2, 3, 4, 5}
```

- Многомерные массивы:

```
var matrix [3][4]int // матрица 3x4, массив массивов
```

Ключи (индексы) массивов

Индексы массива начинаются с 0 и идут до `len(arr)-1`. Попытка доступа к индексу вне этого диапазона вызовет ошибку времени выполнения.

```
value := arr[0] // доступ к первому элементу
arr[4] = 10     // изменение пятого элемента
// arr[5] = 10  // ошибка, индекс вне диапазона
```

Получение и изменение элементов по индексу

Для доступа к элементу массива по индексу используется синтаксис `array[index]`.

```
value := arr[0] // получение значения первого элемента
arr[1] = 10     // изменение значения второго элемента
```

Длина (len) и вместимость (cap) массива

Длина массива - это количество элементов в массиве. Вместимость - это максимальное количество элементов, которое может содержать массив. При изменении длины массива для массива длина и вместимость всегда равны и могут быть получены с помощью функций `len` и `cap`.

```
length := len(arr) // длина массива
capacity := cap(arr) // вместимость массива
```

Итерация по массиву

Итерация по массиву осуществляется с помощью цикла `for`:

```
for i := 0; i < len(arr); i++ {
    fmt.Println(arr[i])
}

// или с использованием конструкции for range:
for i, v := range arr {
    fmt.Println(i, v)
}
```

Что такое срез

Срез (slice) – это динамическая последовательность элементов одного типа, длина и вместимость которой могут изменяться. Срезы являются ссылочными типами и ссылаются на массивы. В отличие от массивов, срезы более гибкие и их длина может изменяться.

```
slice := []int{1, 2, 3}
```

Создание слайса

Срезы можно создавать несколькими способами:

- На основе массива:

```
arr := [5]int{1, 2, 3, 4, 5}
slice := arr[1:4] // срез содержит элементы со 2-го по 4-й (индексы 1, 2, 3)
```

- С помощью литерала:

```
slice := []int{1, 2, 3}
```

- Пустой срез:

```
var emptySlice []int
```

Длина и вместимость слайса

Длина среза (len) – это количество элементов в срезе. Вместимость (cap) – это количество элементов, которые могут быть помещены в срез без выделения новой памяти.

```
slice := []int{1, 2, 3, 4, 5}
fmt.Println(len(slice)) // 5
fmt.Println(cap(slice)) // 5
```

Создание слайса через make

Срезы можно создавать с помощью функции make:

```
slice := make([]int, 5) // создание среза длиной 5 и вместимостью 5
slice2 := make([]int, 5, 10) // создание среза длиной 5 и вместимостью 10
```

Функция append

Функция append добавляет новые элементы в конец среза и возвращает новый срез. Если вместимость среза недостаточна, append выделяет новый массив, копирует в него старые элементы и добавляет новые.

```
slice := []int{1, 2, 3}
slice = append(slice, 4, 5) // добавление элементов 4 и 5
```

Конкатенация срезов

Конкатенация срезов – это добавление двух или более срезов в один срез. Конкатенация срезов возвращает новый срез. Если вместимость среза недостаточна, конкатенация выделяет новый массив, копирует в него старые элементы и добавляет новые.

```
slice1 := []int{1, 2, 3}
slice2 := []int{4, 5}
slice3 := append(slice1, slice2...) // добавление среза slice2 в срез slice1
```

Операция слайсинг (создание среза на основе массива или слайса)

Создание среза на основе массива или другого среза можно выполнить с использованием синтаксиса `slice[start:end]`. Начальный индекс включается, конечный – нет.

```
arr := [5]int{1, 2, 3, 4, 5}
slice := arr[1:4] // срез содержит элементы со 2-го по 4-й (индексы 1, 2, 3)
fmt.Println(slice) // [2 3 4]

slice2 := slice[1:2] // срез на основе другого среза
fmt.Println(slice2) // [3]
```

Функция `copy`

Функция `copy` копирует элементы из одного среза в другой и возвращает количество скопированных элементов. Целевой срез должен иметь достаточную длину для размещения копируемых элементов.

```
src := []int{1, 2, 3}
dst := make([]int, len(src))
n := copy(dst, src) // копирование всех элементов из src в dst
fmt.Println(dst) // [1 2 3]
fmt.Println(n) // 3
```

Дополнительные аспекты массивов

Инициализация массива с использованием ключей (индексов)

Можно инициализировать массив, указывая значения для определённых индексов:

```
arr := [5]int{0: 10, 4: 20} // массив, где первый элемент равен 10, а пятый - 20
```

Копирование массива

В отличие от срезов, при присваивании одного массива другому происходит копирование значений:

```
arr1 := [3]int{1, 2, 3}
arr2 := arr1 // копирование значений
arr2[0] = 10
fmt.Println(arr1) // [1 2 3]
fmt.Println(arr2) // [10 2 3]
```

Дополнительные аспекты срезов

Срезы и память

Срезы являются ссылками на память, поэтому изменения в одном срезе влияют на другие.

Срезы занимают три поля в памяти:

- указатель на начало среза
- длина среза
- вместимость среза

```
slice := []int{1, 2, 3, 4, 5}
slice2 := slice[1:3] // длина 2, вместимость 4
fmt.Println(len(slice2), cap(slice2)) // 2 4
```

Изменение среза изменяет исходный массив

Поскольку срезы ссылаются на массив, изменение среза изменяет массив:

```
arr := [5]int{1, 2, 3, 4, 5}
slice := arr[1:4]
slice[0] = 10
fmt.Println(arr) // [1 10 3 4 5]
```

Nil срезы

Nil срезы эквивалентны пустым срезам, но у них нулевая длина и вместимость:

```
var nilSlice []int
fmt.Println(nilSlice == nil) // true
fmt.Println(len(nilSlice), cap(nilSlice)) // 0 0
```

Срез с максимальной вместимостью

Срез можно создать с указанием максимальной вместимости:

```
arr := [5]int{1, 2, 3, 4, 5}
slice := arr[1:3:4] // длина 2, вместимость 3
fmt.Println(len(slice), cap(slice)) // 2 3
```

Удаление элемента из среза

Удаление элемента из среза требует создания нового среза:

```
slice := []int{1, 2, 3, 4, 5}
index := 2 // индекс элемента для удаления
slice = append(slice[:index], slice[index+1:]...)
fmt.Println(slice) // [1 2 4 5]
```

Вставка элемента в срез

Для вставки элемента в срез требуется создание нового среза:

```
slice := []int{1, 2, 3, 5}
index := 3 // индекс для вставки
value := 4
```

```
slice = append(slice[:index], append([]int{value}, slice[index:]...)...)
fmt.Println(slice) // [1 2 3 4 5]
```

Общая формула вычисления ёмкости среза

Если используется синтаксис с тремя параметрами `arr[low:high:max]`, то ёмкость вычисляется как:

`capacity = max - low`

Если используется синтаксис с двумя параметрами `arr[low:high]`, то ёмкость вычисляется как:

`capacity=len(arr)-low`

Общая формула вычисления ёмкости для пустого среза

Если пустой срез создается с использованием функции `make` с заданной ёмкостью, то ёмкость среза равна указанному значению. Если ёмкость не указана, то по умолчанию она равна 0.

Примеры:

- `make([]int, 0, 10)` – ёмкость 10.
- `var slice []int` – ёмкость 0.
- `[]int{}` – ёмкость 0.

Алгоритм увеличения ёмкости

Go использует алгоритм для увеличения ёмкости среза, который стремится минимизировать количество выделений памяти. Обычно ёмкость увеличивается следующим образом:

- Если ёмкость среза была 0, то она увеличивается до 1.
- Если ёмкость среза была ненулевая, то она обычно удваивается (Для маленьких срезов до ёмкости 1024 элементов).
- Для больших срезов (с ёмкостью более 1024 элементов) ёмкость увеличивается на примерно 25% при каждом добавлении.

Сортировка массива и среза в Go

В Go стандартная библиотека предоставляет функции для сортировки срезов, но не для массивов напрямую.

Однако можно использовать те же функции для сортировки массивов, преобразовав их в срезы.

Основные виды сортировок включают:

- Сортировка пузырьком (Bubble Sort)
- Сортировка вставками (Insertion Sort)
- Сортировка выбором (Selection Sort)
- Сортировка слиянием (Merge Sort)
- Сортировка быстрая (Quick Sort)
- Сортировка кучей (Heap Sort)

Также для поиска элементов в отсортированных и неотсортированных коллекциях используются:

- Бинарный поиск (Binary Search)
- Линейный поиск (Linear Search)

Сортировка пузырьком (Bubble Sort)

Сортировка пузырьком – простой алгоритм сортировки, который многократно проходит через массив, сравнивая соседние элементы и меняя их местами, если они находятся в неправильном порядке.

Проход по массиву повторяется до тех пор, пока не будут выполнены все перестановки.

Сложность:

- Best: $O(n)$
- Average: $O(n^2)$
- Worst: $O(n^2)$

```
func bubbleSort(slice []int) {
    n := len(slice)
    for i := 0; i < n-1; i++ {
        for j := 0; j < n-i-1; j++ {
            if slice[j] > slice[j+1] {
                slice[j], slice[j+1] = slice[j+1], slice[j]
            }
        }
    }
}

func main() {
    slice := []int{64, 34, 25, 12, 22, 11, 90}
    bubbleSort(slice)
    fmt.Println("Bubble Sort:", slice) // [11 12 22 25 34 64 90]
}
```

Сортировка вставками (Insertion Sort)

Сортировка вставками строит отсортированную часть массива по одному элементу за раз. Каждый элемент вставляется в свою позицию в уже отсортированной части.

Сложность:

- Best: $O(n)$
- Average: $O(n^2)$
- Worst: $O(n^2)$

```
func insertionSort(slice []int) {
    for i := 1; i < len(slice); i++ {
        key := slice[i]
        j := i - 1
        for j >= 0 && slice[j] > key {
            slice[j+1] = slice[j]
            j--
        }
        slice[j+1] = key
    }
}

func main() {
    slice := []int{64, 34, 25, 12, 22, 11, 90}
    insertionSort(slice)
    fmt.Println("Insertion Sort:", slice) // [11 12 22 25 34 64 90]
}
```

Сортировка выбором (Selection Sort)

Сортировка выбором находит минимальный элемент из неотсортированной части массива и меняет его местами с первым элементом неотсортированной части. Затем процесс повторяется для оставшейся части.

Сложность:

- Best: $O(n^2)$
- Average: $O(n^2)$
- Worst: $O(n^2)$

```
func selectionSort(slice []int) {
    n := len(slice)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if slice[j] < slice[minIdx] {
                minIdx = j
            }
        }
        slice[i], slice[minIdx] = slice[minIdx], slice[i]
    }
}

func main() {
    slice := []int{64, 34, 25, 12, 22, 11, 90}
    selectionSort(slice)
    fmt.Println("Selection Sort:", slice) // [11 12 22 25 34 64 90]
}
```

Сортировка слиянием (Merge Sort)

Сортировка слиянием рекурсивно делит массив на две половины, сортирует каждую половину и затем сливает отсортированные половины в один отсортированный массив.

Сложность:

- Best: $O(n \log(n))$
- Average: $O(n \log(n))$
- Worst: $O(n \log(n))$

```
func merge(left, right []int) []int {
    result := []int{}
    i, j := 0, 0
    for i < len(left) && j < len(right) {
        if left[i] < right[j] {
            result = append(result, left[i])
            i++
        } else {
            result = append(result, right[j])
            j++
        }
    }
    result = append(result, left[i:]...)
    result = append(result, right[j:]...)
    return result
}
```

```

func mergeSort(slice []int) []int {
    if len(slice) <= 1 {
        return slice
    }
    mid := len(slice) / 2
    left := mergeSort(slice[:mid])
    right := mergeSort(slice[mid:])
    return merge(left, right)
}

func main() {
    slice := []int{64, 34, 25, 12, 22, 11, 90}
    sortedSlice := mergeSort(slice)
    fmt.Println("Merge Sort:", sortedSlice) // [11 12 22 25 34 64 90]
}

```

Быстрая сортировка (Quick Sort)

Быстрая сортировка выбирает опорный элемент, разделяет массив на элементы меньше и больше опорного, и рекурсивно сортирует обе части.

Сложность:

- Best: $O(n \log(n))$
- Average: $O(n \log(n))$
- Worst: $O(n^2)$

```

func quickSort(slice []int) []int {
    if len(slice) < 2 {
        return slice
    }
    pivot := slice[len(slice)/2]
    left := []int{}
    right := []int{}
    for _, v := range slice {
        if v < pivot {
            left = append(left, v)
        } else if v > pivot {
            right = append(right, v)
        }
    }
    return append(append(quickSort(left), pivot), quickSort(right)...)
}

func main() {
    slice := []int{64, 34, 25, 12, 22, 11, 90}
    sortedSlice := quickSort(slice)
    fmt.Println("Quick Sort:", sortedSlice) // [11 12 22 25 34 64 90]
}

```

Сортировка кучей (Heap Sort)

Сортировка кучей строит структуру данных "куча" и затем извлекает элементы из этой кучи, чтобы получить отсортированный массив.

Сложность:

- Best: $O(n \log(n))$


```

- Average:  $O(n \log(n))$ 
- Worst:  $O(n \log(n))$ 

func heapify(slice []int, n, i int) {
    largest := i
    left := 2*i + 1
    right := 2*i + 2
    if left < n && slice[left] > slice[largest] {
        largest = left
    }
    if right < n && slice[right] > slice[largest] {
        largest = right
    }
    if largest != i {
        slice[i], slice[largest] = slice[largest], slice[i]
        heapify(slice, n, largest)
    }
}

func heapSort(slice []int) {
    n := len(slice)
    for i := n / 2 - 1; i >= 0; i-- {
        heapify(slice, n, i)
    }
    for i := n - 1; i >= 0; i-- {
        slice[0], slice[i] = slice[i], slice[0]
        heapify(slice, i, 0)
    }
}

func main() {
    slice := []int{64, 34, 25, 12, 22, 11, 90}
    heapSort(slice)
    fmt.Println("Heap Sort:", slice) // [11 12 22 25 34 64 90]
}

```

Поиск элементов

Линейный поиск (Linear Search)

Линейный поиск проходит через все элементы массива или среза, пока не найдет искомый элемент.

Сложность:

- Best: $O(1)$
- Average: $O(n)$
- Worst: $O(n)$

```

func linearSearch(slice []int, target int) int {
    for i, v := range slice {
        if v == target {
            return i
        }
    }
    return -1 // элемент не найден
}

```

```
func main() {
    slice := []int{10, 20, 30, 40, 50}
    index := linearSearch(slice, 30)
    fmt.Println("Index of 30:", index) // Index of 30: 2
}
```

Бинарный поиск (Binary Search)

Бинарный поиск работает на отсортированных массивах и делит массив на две половины, рекурсивно сокращая диапазон поиска.

Сложность:

- Best: $O(\log(n))$
- Average: $O(\log(n))$
- Worst: $O(\log(n))$

```
func binarySearch(slice []int, target int) int {
    low, high := 0, len(slice)-1
    for low <= high {
        mid := low + (high-low)/2
        if slice[mid] == target {
            return mid
        }
        if slice[mid] < target {
            low = mid + 1
        } else {
            high = mid - 1
        }
    }
    return -1 // элемент не найден
}
```

```
func main() {
    slice := []int{10, 20, 30, 40, 50}
    index := binarySearch(slice, 30)
    fmt.Println("Index of 30:", index) // Index of 30: 2
}
```

Итоги по сортировкам и поискам

- Сортировка пузырьком, вставками и выбором лучше подходят для учебных целей или очень небольших массивов, так как их сложность $O(n^2)$ делает их неэффективными для больших данных.
- Сортировка слиянием и быстрая сортировка являются более эффективными для больших массивов с временем выполнения $O(n \log(n))$
- Сортировка кучей также эффективна и подходит для ситуаций, где требуется стабильная производительность в худшем случае.
- Линейный поиск применим к неотсортированным данным, а бинарный поиск эффективен для отсортированных данных, обеспечивая быструю скорость поиска.

sort пакет

В Go стандартный пакет `sort` предоставляет функции для сортировки срезов. Он поддерживает сортировку срезов чисел и строк, а также может быть использован для сортировки пользовательских типов через интерфейс `sort.Interface`.

Основные функции

- `sort.Ints(slice []int)`
 - Сортирует срез целых чисел в порядке возрастания.

```
slice := []int{4, 2, 3, 1}
sort.Ints(slice)
fmt.Println(slice) // [1 2 3 4]
```

- `sort.Float64s(slice []float64)`
 - Сортирует срез вещественных чисел в порядке возрастания.

```
slice := []float64{4.1, 2.2, 3.3, 1.4}
sort.Float64s(slice)
fmt.Println(slice) // [1.4 2.2 3.3 4.1]
```

- `sort.Strings(slice []string)`
 - Сортирует срез строк в порядке возрастания.

```
slice := []string{"dog", "cat", "bird"}
sort.Strings(slice)
fmt.Println(slice) // [bird cat dog]
```

- `sort.Sort(sort.Interface)`
 - Позволяет сортировать срезы пользовательских типов, реализующих интерфейс `sort.Interface`. Интерфейс требует реализации трех методов: `Len()`, `Less(i, j int) bool`, и `Swap(i, j int)`.

```
type Person struct {
    Name string
    Age  int
}

type ByAge []Person

func (a ByAge) Len() int           { return len(a) }
func (a ByAge) Less(i, j int) bool { return a[i].Age < a[j].Age }
func (a ByAge) Swap(i, j int)      { a[i], a[j] = a[j], a[i] }

people := []Person{
    {"Alice", 30},
    {"Bob", 25},
    {"Charlie", 35},
}

sort.Sort(ByAge(people))
fmt.Println(people) // [{Bob 25} {Alice 30} {Charlie 35}]
```

Встроенные возможности Go для поиска

Поиск в срезах

Go стандартная библиотека не предоставляет встроенные функции для бинарного поиска в срезах, но можно использовать функции из пакета `sort` для этого.

Основные функции:

- `sort.SearchInts(slice []int, x int) int`
 - Ищет значение `x` в отсортированном срезе целых чисел и возвращает индекс первого элемента, равного `x`, или индекс, где элемент мог бы быть вставлен, если `x` отсутствует.

```
slice := []int{1, 2, 3, 4, 5}
index := sort.SearchInts(slice, 3)
fmt.Println(index) // 2
```

- `sort.SearchFloat64s(slice []float64, x float64) int`
 - Ищет значение `x` в отсортированном срезе вещественных чисел.

```
slice := []float64{1.1, 2.2, 3.3, 4.4, 5.5}
index := sort.SearchFloat64s(slice, 3.3)
fmt.Println(index) // 2
```

- `sort.SearchStrings(slice []string, x string) int`
 - Ищет строку `x` в отсортированном срезе строк.

```
slice := []string{"apple", "banana", "cherry"}
index := sort.SearchStrings(slice, "banana")
fmt.Println(index) // 1
```